

Pre-trained CNN Representation Transfer and Robustness Analysis

GNR638: Machine Learning for Remote Sensing II

Group 28

Team Members: 22B3932, 23B4207

Spring 2025-26

Contents

1	Introduction	3
1.1	Transfer Learning in Computer Vision	3
1.2	Remote Sensing and Aerial Image Classification	3
1.3	Objective and Scope	4
2	Experimental Framework	4
2.1	Project Architecture and Reproducibility	4
2.1.1	Package Configuration	5
2.1.2	Directory Structure	5
2.1.3	Implementation Modules	5
2.2	Dataset: Aerial Image Dataset (AID)	6
2.2.1	Dataset Composition	6
2.2.2	Data Preprocessing and Augmentation	7
2.2.3	Dataset Splitting and Reproducibility	7
2.3	Model Architectures	7
2.3.1	Efficiency Metrics	8
2.4	Training Setup	8
2.4.1	Optimizer and Loss Function	8
2.4.2	Hyperparameters	9
2.4.3	Epoch Limits	9
2.4.4	Reproducibility	9
3	Scenario 1: Linear Probe Transfer	9
3.1	Method	9
3.2	Results	10
3.3	Training and Validation Accuracy Curves	11
3.4	Visual Analysis: Feature Embedding Visualization	12
3.4.1	PCA Visualization	12
3.4.2	t-SNE Visualization	13

3.5	Confusion Matrix	14
3.6	Analysis and Insight	14
4	Scenario 2: Fine-Tuning Strategies	15
4.1	Method	15
4.2	Trainable Parameters per Strategy	16
4.3	Accuracy vs Percentage of Unfrozen Parameters	16
4.4	Convergence Stability: Training Loss vs Epoch	17
4.5	Gradient Norm Statistics	18
4.6	Analysis	18
5	Scenario 3: Few-Shot Learning Analysis	19
5.1	Method	19
5.2	Results	20
5.3	Validation Accuracy Across Epochs	21
5.4	Training-Validation Accuracy Gap	22
5.5	Accuracy vs Data Fraction	23
5.6	Analysis	23
6	Scenario 4: Corruption Robustness Evaluation	24
6.1	Method	24
6.2	Results	25
6.3	Analysis	25
7	Scenario 5: Layer-Wise Feature Probing	26
7.1	Method	26
7.1.1	Layer Selection	26
7.2	Feature Norm Statistics	26
7.3	Validation Accuracy vs Depth	27
7.4	PCA Visualization Across Layers	28
7.4.1	ResNet50	28
7.4.2	DenseNet121	28
7.4.3	EfficientNet-B0	28
7.5	Analysis	29
8	Discussion and Key Findings	30
8.1	Which Architecture Transfers Best and Why?	30
8.2	Which Model is Most Robust Under Distribution Shift?	30
8.3	The Impact of Inductive Bias	30
8.4	When Does Fine-Tuning Degrade Robustness?	31
8.5	Negative and Failure Case	31
9	Conclusion and Computational Budget	32
9.1	Summary of Findings	32
9.2	Computational Budget	33

Abstract

This report presents a systematic evaluation of three ImageNet pre-trained CNN backbones — ResNet50, DenseNet121, and EfficientNet-B0 — on the Aerial Image Dataset (AID), a 30-class remote sensing classification benchmark. Five experimental scenarios are studied: linear probe transfer, fine-tuning strategies, few-shot learning, corruption robustness, and layer-wise feature probing.

1 Introduction

1.1 Transfer Learning in Computer Vision

Transfer learning has become the dominant paradigm in deep learning based computer vision. Rather than training deep neural networks from scratch on a target dataset, models are initialized with weights pre-trained on large scale datasets — most commonly ImageNet, which contains over 1.2 million images across 1000 categories. These pre-trained weights encode a rich hierarchy of visual features, from low-level edges and textures in early layers to high-level object parts and semantic concepts in deeper layers. By transferring these representations to a new task, practitioners can achieve strong performance even with limited labeled data and significantly reduced training time.

Two primary transfer learning strategies are widely used in practice:

- **Linear Probe Transfer** — The pre-trained backbone is used as a frozen feature extractor and only a new linear classification head is trained on the target dataset. This strategy evaluates the quality of the pre-trained representations directly, without any task-specific adaptation.
- **Fine-Tuning** — Some or all of the backbone parameters are unfrozen and updated jointly with the classification head on the target dataset. This allows the representations to adapt to the statistics of the new domain, typically yielding higher accuracy at the cost of increased training time and potential loss of generalization.

1.2 Remote Sensing and Aerial Image Classification

Remote sensing image classification presents unique challenges that distinguish it from standard natural image recognition tasks. Aerial images captured from satellites or aircraft differ fundamentally from ground-level photographs in several ways:

- **Top-down perspective** — Objects are viewed from above, eliminating the familiar viewpoint cues present in natural images. Scene categories are determined by spatial arrangements of objects rather than the objects themselves.
- **Scale variation** — Aerial images span a wide range of spatial resolutions, from sub-meter to tens of meters per pixel, introducing significant intra-class variation.

- **Domain gap** — ImageNet contains ground-level photographs of objects and animals. The visual statistics of aerial imagery — texture patterns, color distributions, spatial layouts — differ substantially from ImageNet, raising the question of how well ImageNet pre-trained features transfer to this domain.

These characteristics make aerial image classification an ideal testbed for studying representation transfer, as the domain gap between ImageNet and aerial imagery is large enough to reveal meaningful differences between backbone architectures and fine-tuning strategies.

1.3 Objective and Scope

The objective of this assignment is to systematically analyze how different pre-trained CNN backbones behave when transferred to aerial image classification under controlled experimental conditions. Specifically, this study investigates:

- How well do frozen ImageNet features transfer to aerial imagery without any adaptation?
- How do different fine-tuning strategies affect performance, convergence, and gradient dynamics?
- Which architecture is most data-efficient under limited supervision?
- How robust are fine-tuned models to input corruptions such as Gaussian noise, motion blur, and brightness shifts?
- How does the quality of learned representations evolve across network depth?

Three backbone architectures are compared — ResNet50, DenseNet121, and EfficientNet-B0 — each representing a distinct design philosophy in CNN development. All models are pre-trained on ImageNet and evaluated on the Aerial Image Dataset (AID), a benchmark containing 10,000 aerial scene images across 30 categories. All experiments are conducted under a unified and reproducible framework to ensure fair comparison across architectures and scenarios.

2 Experimental Framework

2.1 Project Architecture and Reproducibility

To ensure a structured and reproducible experimental workflow, the project was organized as an installable Python package using a `pyproject.toml` build configuration. The package, named `cnn-transfer`, is defined under the `src/` layout and can be installed in editable mode via `pip install -e .`, ensuring that all notebooks share the same versioned codebase without path manipulation. The implementation separates dataset handling, model configuration, training, evaluation, and analysis into independent modules, allowing the same experimental pipeline to be reused across all three CNN backbones without code duplication.

2.1.1 Package Configuration

The project uses `pyproject.toml` with `setuptools` as the build backend. All dependencies are declared explicitly, including `torch`, `torchvision`, `timm`, `ptflops`, `scikit-learn`, `umap-learn`, `tqdm`, and `matplotlib`. This ensures that the experimental environment can be exactly reproduced from a single configuration file.

2.1.2 Directory Structure

The project workspace is organized as follows:

```
src/cnn_transfer/
    Core implementation of the transfer learning framework

notebooks/
    experiments_resnet50.ipynb
    experiments_densenet121.ipynb
    experiments_efficientnet_b0.ipynb

results/
    CSV files summarizing model performance

plots/
    Training curves, PCA, t-SNE, UMAP visualizations,
    and robustness analysis plots

checkpoints/
    Saved model weights and training checkpoints

logs/
    Training logs and experiment tracking information
```

This structure allows experiments for different architectures to be organized independently while sharing the same underlying framework.

2.1.3 Implementation Modules

The core functionality is implemented through the following modules, all residing under `src/cnn_transfer/`. Each module has a single well-defined responsibility, and all three experiment notebooks import from the same installed package, guaranteeing identical behavior across architectures.

Module	Primary Responsibility
<code>dataset.py</code>	Dataset loading, preprocessing, train-validation split, and few-shot subset sampling.
<code>models.py</code>	Loading pretrained backbones via <code>timm</code> and implementing freezing/unfreezing strategies for transfer learning.
<code>train.py</code>	Training loop, validation procedure, and per-epoch gradient norm tracking.
<code>evaluate.py</code>	Evaluation metrics, confusion matrices, and feature visualizations (PCA, t-SNE, UMAP).
<code>corruption.py</code>	Image corruption functions (Gaussian noise, motion blur, brightness shift) and robustness evaluation utilities.
<code>feature_probe.py</code>	Forward hook-based intermediate feature extraction and linear probe training for layer-wise analysis.
<code>utils.py</code>	Parameter counting, FLOPs/MACs computation via <code>ptflops</code> , and global reproducibility seed setting.

Table 1: Core implementation modules of the `cnn-transfer` package.

This modular architecture ensures that all models are trained and evaluated using a consistent and reproducible experimental pipeline, enabling fair comparison across architectures and scenarios.

2.2 Dataset: Aerial Image Dataset (AID)

The Aerial Image Dataset (AID) is a benchmark dataset designed for aerial scene classification using remote sensing imagery. Unlike natural image datasets such as ImageNet, AID contains aerial views where scene categories are determined by spatial arrangements of objects such as buildings, roads, vegetation, and water bodies.

2.2.1 Dataset Composition

The dataset has the following characteristics:

- **Total Images:** 6993 RGB images
- **Number of Classes:** 30 scene categories
- **Class Distribution:** Approximately 220–420 images per class
- **Example Classes:** Airport, Bridge, Forest, Meadow, Industrial Area, Viaduct, Storage Tanks
- **Image Source:** Collected from Google Earth imagery
- **Original Resolution:** Approximately 600×600 pixels
- **Spatial Resolution:** Ranges from 0.5 m to 8 m per pixel, introducing scale variation across scenes

2.2.2 Data Preprocessing and Augmentation

To match the input requirements of the pre-trained CNN models, the following preprocessing pipeline was applied:

- **Resizing:** All images were resized to 224×224 pixels.
- **Normalization:** Images were normalized using ImageNet statistics:
 - Mean: $[0.485, 0.456, 0.406]$
 - Standard Deviation: $[0.229, 0.224, 0.225]$
- **Data Augmentation:** Random horizontal flipping ($p = 0.5$) was applied to training images only to improve generalization and introduce orientation invariance.

2.2.3 Dataset Splitting and Reproducibility

To ensure consistent evaluation across all experiments, the dataset was split as follows:

- **Training Set:** 85% of the dataset
- **Validation Set:** 15% of the dataset
- **Random Seed:** A fixed seed (42) was used to ensure reproducibility of the split.

For the few-shot learning experiments, additional subsets of the training data were created:

- **100% Training Data:** Full dataset training
- **20% Training Data:** Reduced training subset
- **5% Training Data:** Extreme low-data scenario

These subsets were generated using deterministic sampling to ensure that all models were evaluated on identical training subsets.

2.3 Model Architectures

Three ImageNet pre-trained CNN backbones were selected for this study: ResNet50, DenseNet121, and EfficientNet-B0. Each architecture represents a distinct design philosophy in convolutional neural network development.

ResNet50 introduced residual (skip) connections that allow gradients to bypass convolutional blocks during backpropagation. This alleviates the vanishing gradient problem and enables training of much deeper networks. ResNet50 consists of 4 residual stages with bottleneck blocks, producing a 2048-dimensional feature vector before the classification head.

DenseNet121 uses dense connectivity where each layer receives feature maps from all preceding layers within a dense block. This dense feature reuse encourages the network to learn compact and non-redundant representations while improving gradient flow. DenseNet121 contains four dense blocks with a final 1024-dimensional feature vector.

EfficientNet-B0 is built on a compound scaling strategy that jointly scales network depth, width, and input resolution using a fixed scaling coefficient. Its architecture is composed of MBConv (Mobile Inverted Bottleneck Convolution) blocks with depthwise separable convolutions and Squeeze-and-Excitation (SE) attention modules, making it highly parameter-efficient while maintaining competitive accuracy.

2.3.1 Efficiency Metrics

All three models were profiled using the `ptflops` library with input size $(3 \times 224 \times 224)$. MACs (Multiply-Accumulate Operations) measure the number of arithmetic operations per forward pass, while FLOPs are reported as $2 \times$ MACs following standard convention.

Model	Parameters	MACs	FLOPs
ResNet50	23.57 M	4.13 G	8.26 G
DenseNet121	6.98 M	2.87 G	5.74 G
EfficientNet-B0	4.05 M	0.39 G	0.78 G

Table 2: Efficiency metrics for the three selected backbones, measured at input resolution 224×224 .

EfficientNet-B0 is the most computationally efficient model, requiring approximately one-tenth the FLOPs of ResNet50 while using only 17% of its parameters. DenseNet121 occupies a middle ground with moderate parameter count but relatively high MACs due to its dense connectivity pattern. ResNet50 is the largest and most computationally expensive backbone, but benefits from the strongest absolute representational capacity at its scale.

2.4 Training Setup

All experiments across the three backbones were conducted under a unified training configuration to ensure fair and controlled comparison. The following hyperparameters were kept fixed across all scenarios unless explicitly stated otherwise.

2.4.1 Optimizer and Loss Function

The **Adam optimizer** was used for all training runs. Adam was chosen over SGD due to its adaptive learning rate properties, which are particularly beneficial when training with frozen or partially frozen backbones where different parameter groups may have very different gradient magnitudes.

Cross-Entropy Loss was used as the classification objective for all 30-class experiments.

2.4.2 Hyperparameters

Hyperparameter	Value
Optimizer	Adam
Learning Rate	1×10^{-3} (linear probe), 1×10^{-4} (fine-tuning)
Loss Function	Cross-Entropy
Batch Size	64
Image Size	224×224
Num Workers	2
Random Seed	42

Table 3: Fixed hyperparameters used across all experimental scenarios.

2.4.3 Epoch Limits

To respect the computational budget constraints defined in the assignment, the following epoch limits were enforced:

Scenario	Setting	Max Epochs
Linear Probe Transfer	Full data (100%)	20
Fine-Tuning Strategies	Full data (100%)	10
Few-Shot Learning	100%, 20%, 5% training subsets	10
Corruption Robustness	Evaluation only	—
Layer-Wise Probing	Logistic Regression	—

Table 4: Epoch limits per experimental scenario.

2.4.4 Reproducibility

A global seed of 42 was applied at the start of every experiment using a unified `set_seed()` utility that seeds Python’s `random`, NumPy, PyTorch CPU, and PyTorch CUDA simultaneously. DataLoader workers were seeded via a `worker_init_fn` to prevent stochastic variation in data loading order. All train/validation splits and few-shot subsets were generated using fixed `torch.Generator` and `numpy` seeds, ensuring that every model was trained and evaluated on identical data splits.

3 Scenario 1: Linear Probe Transfer

3.1 Method

In this scenario, all backbone parameters were completely frozen and only the final classification head was trained. The original ImageNet classifier was replaced with a new linear layer mapping the backbone’s feature dimension to 30 output classes corresponding to the

AID categories. This setup evaluates the quality of ImageNet pre-trained representations directly, without any task-specific backbone adaptation.

Training was conducted for 20 epochs using the Adam optimizer. The percentage of trainable parameters under this setup is minimal, as shown below:

Model	Trainable Params	Total Params	Trainable %
ResNet50	61,470	23,569,502	0.26%
DenseNet121	30,750	6,984,606	0.44%
EfficientNet-B0	38,430	4,045,978	0.95%

Table 5: Trainable parameters under the linear probe setup.

3.2 Results

The final training and validation accuracies after 20 epochs are reported below.

Model	Final Train Acc	Final Val Acc
ResNet50	0.8604	0.8091
DenseNet121	0.9598	0.8860
EfficientNet-B0	0.8841	0.7993

Table 6: Linear probe final accuracy after 20 epochs on the AID dataset.

DenseNet121 achieves the highest validation accuracy (88.6%), outperforming ResNet50 (80.9%) and EfficientNet-B0 (79.9%) by a significant margin despite having far fewer parameters than ResNet50. This result demonstrates that DenseNet121’s dense connectivity produces more linearly separable feature representations for aerial imagery straight from ImageNet pre-training.

3.3 Training and Validation Accuracy Curves

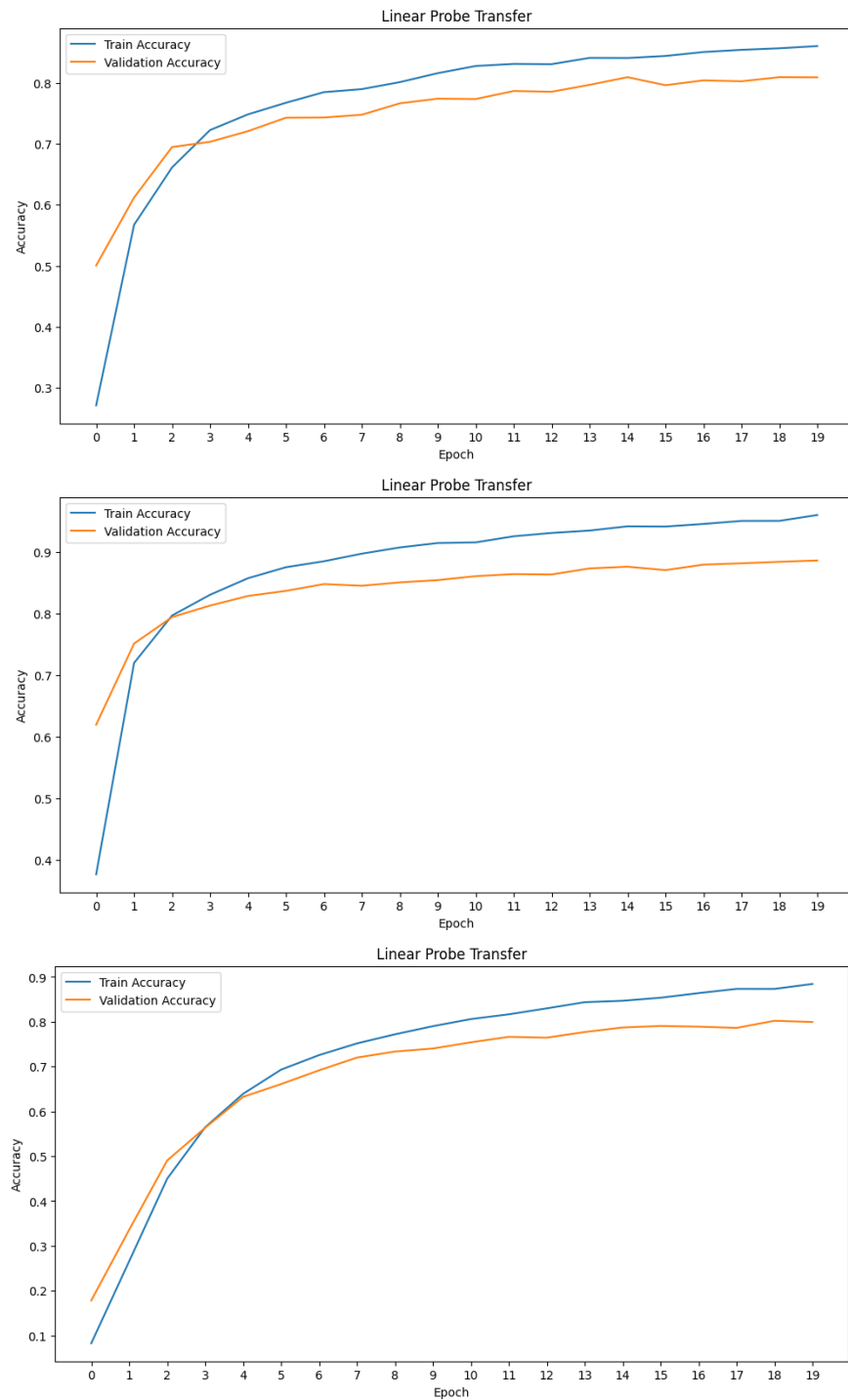


Figure 1: Training and validation accuracy curves for ResNet50 (top), DenseNet121 (middle), and EfficientNet-B0 (bottom) under linear probe training for 20 epochs.

3.4 Visual Analysis: Feature Embedding Visualization

To assess the geometric structure of the frozen backbone features, we extracted the final-layer representations for all validation samples and projected them into 2D using PCA and t-SNE. A well-transferring backbone should produce features where samples of the same aerial scene class cluster tightly and different classes are well-separated, even without any fine-tuning.

3.4.1 PCA Visualization

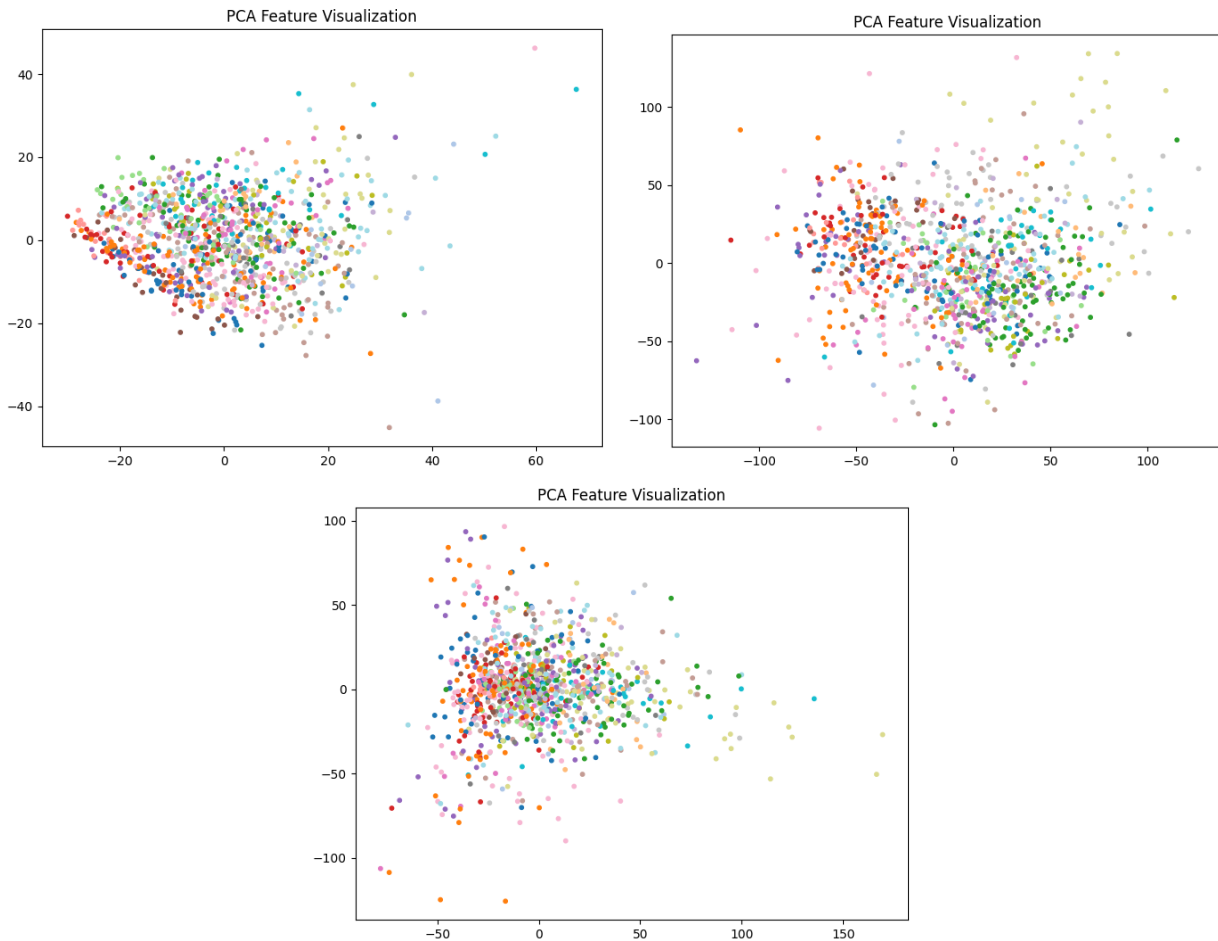


Figure 2: PCA 2D projection of frozen backbone features on the AID validation set. ResNet50 (top left), DenseNet121 (top right), EfficientNet-B0 (bottom).

3.4.2 t-SNE Visualization

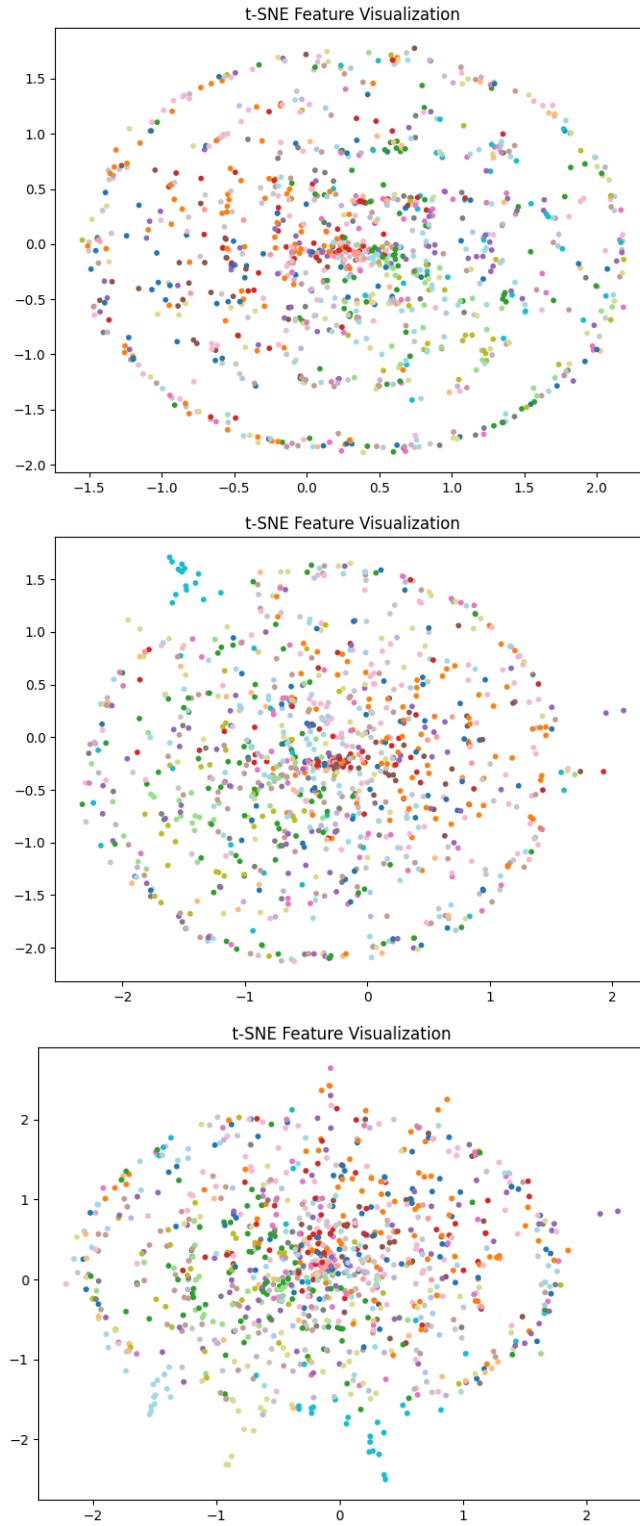


Figure 3: t-SNE 2D projection of frozen backbone features on the AID validation set. ResNet50 (top), DenseNet121 (middle), EfficientNet-B0 (bottom).

3.5 Confusion Matrix

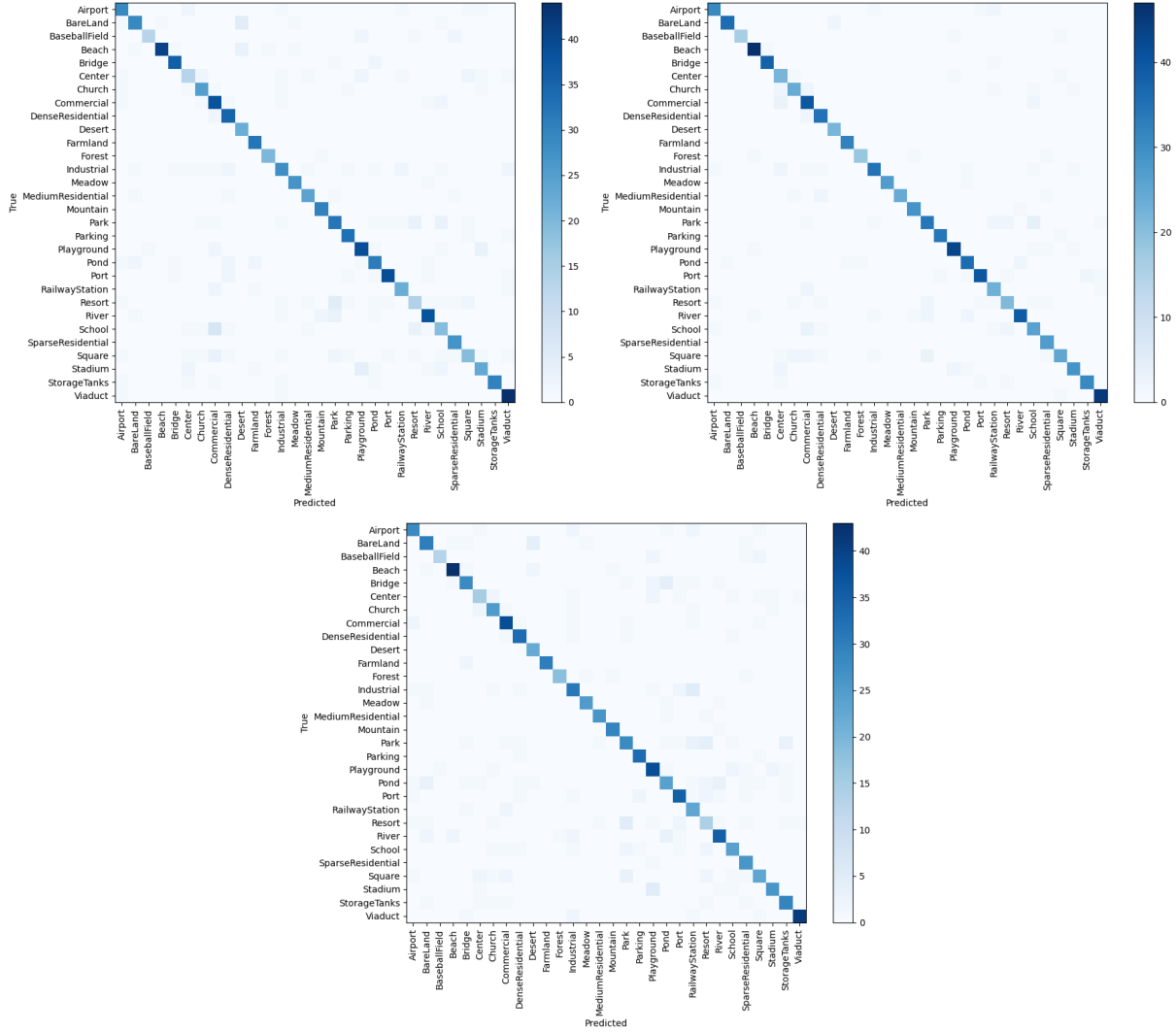


Figure 4: Confusion matrices on the AID validation set under linear probe evaluation. ResNet50 (top left), DenseNet121 (top right), EfficientNet-B0 (bottom).

3.6 Analysis and Insight

Which model has the best out-of-the-box features for aerial imagery? DenseNet121 is the clear winner under the linear probe setting. Its dense skip connections force every layer to preserve and reuse features from all preceding layers, resulting in a final representation that aggregates information across multiple scales and abstraction levels simultaneously. This multi-scale aggregation aligns well with the requirements of aerial scene classification, where both low-level texture cues (vegetation, water, concrete) and high-level spatial arrangements (runway layout, building density) are jointly discriminative.

ResNet50 performs reasonably well (80.9%) but its residual connections only propagate identity shortcuts between adjacent blocks, so the final feature vector is less multi-scale than

DenseNet121’s. EfficientNet-B0 performs the worst under linear probing (79.9%), which is counterintuitive given its strong ImageNet accuracy. This is explained by its use of depthwise separable convolutions: while these are parameter-efficient, they process channels independently and may produce features that are less directly linearly separable without at least partial backbone adaptation.

The PCA projections show partial class separability for all three models, with visually distinct classes such as airports and storage tanks forming recognizable clusters, while spectrally similar classes such as forest and farmland overlap considerably. The t-SNE plots reveal finer cluster structure, with DenseNet121 showing the most compact and well-separated class groupings among the three models, consistent with its higher linear probe accuracy.

Failure case: EfficientNet-B0’s linear probe accuracy lagged behind both ResNet50 and DenseNet121 by approximately 9 percentage points. Despite being the most parameter-efficient model overall, its depthwise separable features require at least partial backbone adaptation to become linearly separable for a new domain. This highlights that computational efficiency and representational transferability are not the same property.

4 Scenario 2: Fine-Tuning Strategies

4.1 Method

In this scenario, four fine-tuning strategies were compared across all three backbones. All strategies were trained for 10 epochs with identical hyperparameters to ensure a fair comparison. The four strategies are:

1. **Linear Probe** — Only the classification head is trained. All backbone parameters remain frozen. This serves as the baseline.
2. **Last Block Fine-Tuning** — The final convolutional block of the backbone is unfrozen along with the classifier head. The unfrozen layers are: `layer4` for ResNet50, `denseblock4` for DenseNet121, and `blocks[-1]` for EfficientNet-B0.
3. **Selective 20% Unfreezing** — Approximately the deepest 20% of parameters by count are unfrozen, prioritizing high-level semantic layers. The rationale is that deeper layers encode task-specific high-level features and benefit most from adaptation, while shallow layers encode general low-level textures that are domain-agnostic and do not require updating.
4. **Full Fine-Tuning** — All backbone parameters are unfrozen and trained jointly with the classification head.

4.2 Trainable Parameters per Strategy

Model	Strategy	% Params	Final Val Acc
ResNet50	Linear Probe	0.26%	0.7623
	Selective 20%	19.21%	0.8971
	Last Block	63.75%	0.9268
	Full Fine-Tuning	100.00%	0.9488
DenseNet121	Linear Probe	0.44%	0.8551
	Selective 20%	19.73%	0.9249
	Last Block	31.34%	0.9295
	Full Fine-Tuning	100.00%	0.9449
EfficientNet-B0	Linear Probe	0.95%	0.7307
	Last Block	18.68%	0.9047
	Selective 20%	20.26%	0.9038
	Full Fine-Tuning	100.00%	0.9415

Table 7: Summary of fine-tuning strategies: percentage of unfrozen parameters and final validation accuracy for all three models.

4.3 Accuracy vs Percentage of Unfrozen Parameters

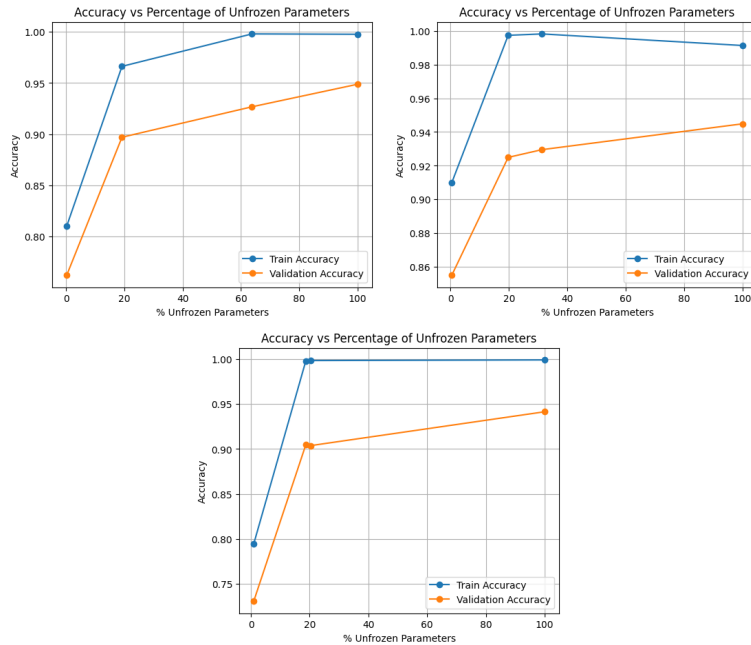


Figure 5: Validation accuracy vs percentage of unfrozen parameters for ResNet50 (top left), DenseNet121 (top right), and EfficientNet-B0 (bottom).

4.4 Convergence Stability: Training Loss vs Epoch

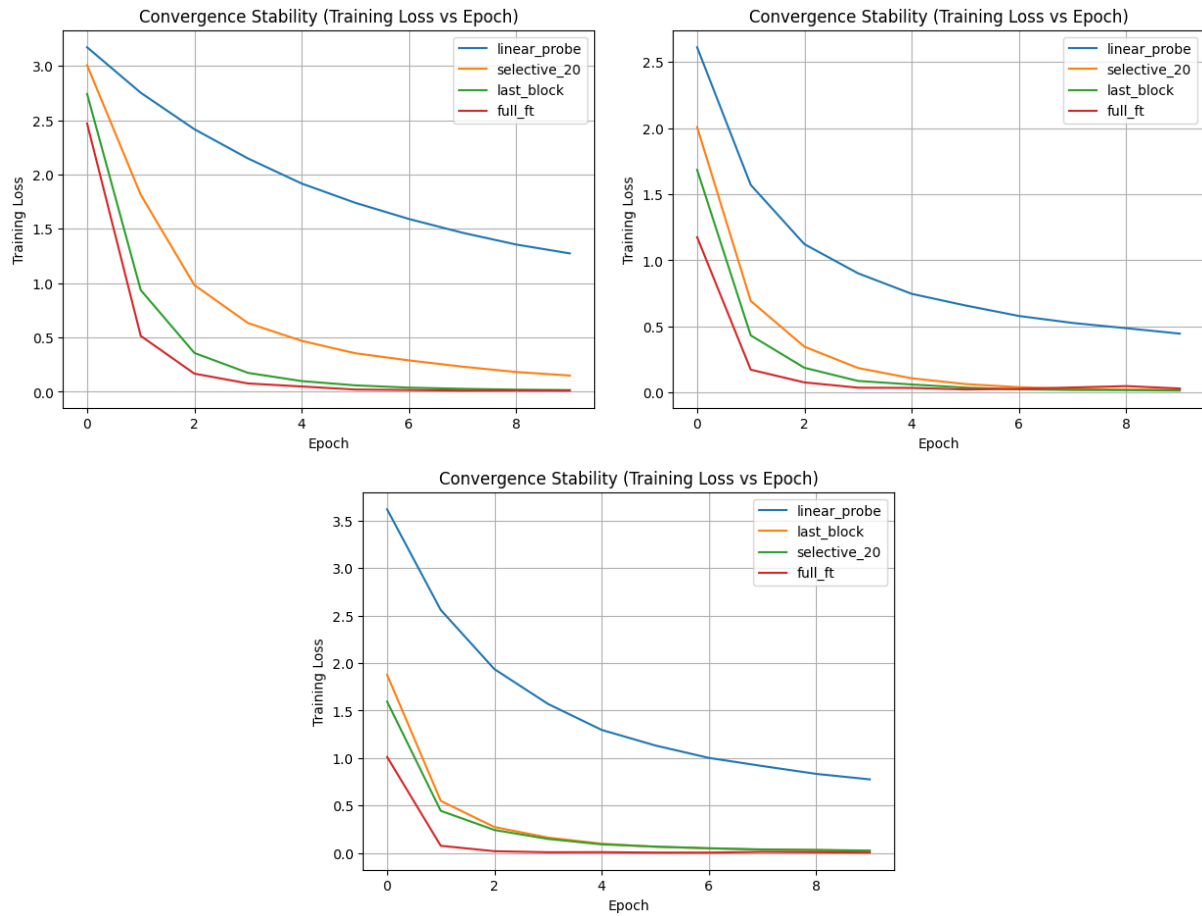


Figure 6: Training loss vs epoch for all four fine-tuning strategies. ResNet50 (top left), DenseNet121 (top right), EfficientNet-B0 (bottom).

4.5 Gradient Norm Statistics

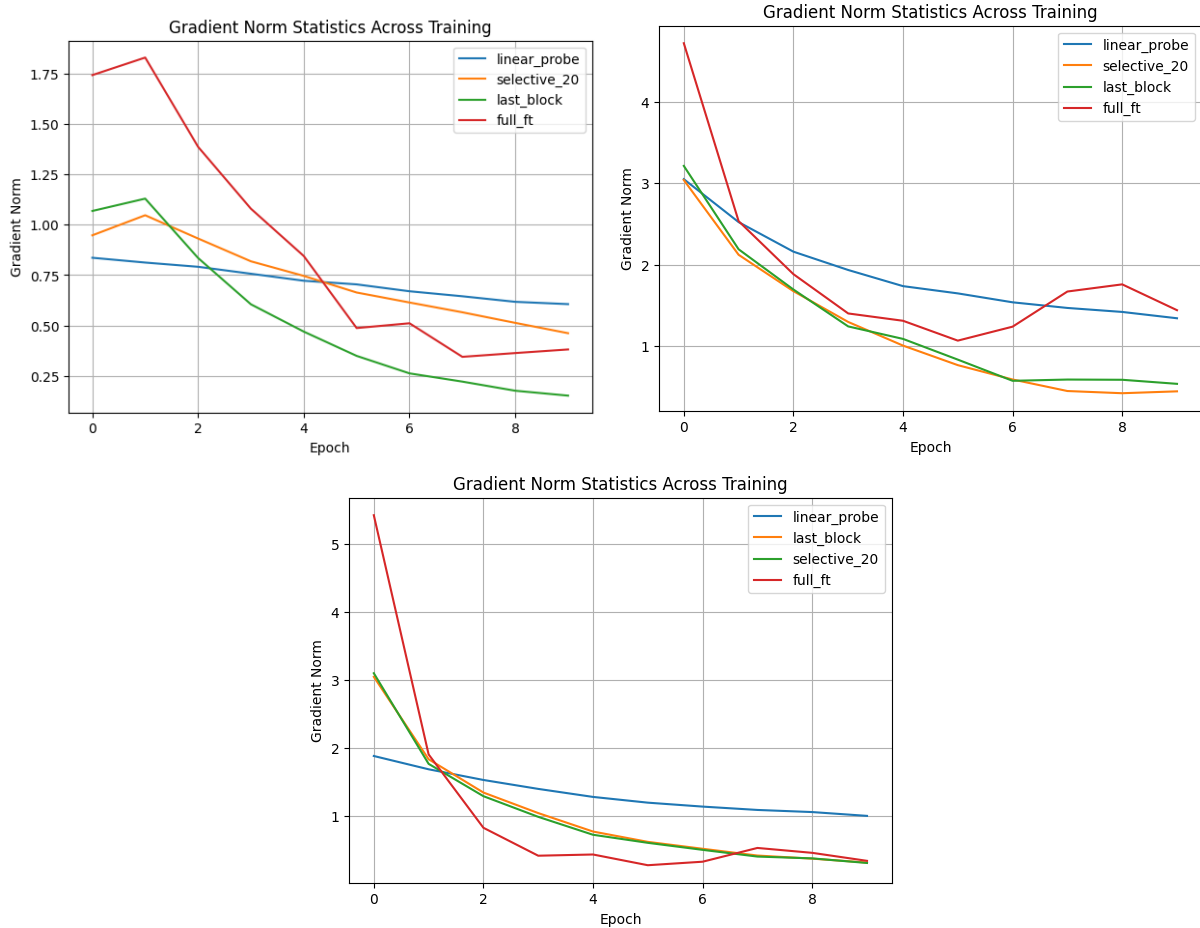


Figure 7: Gradient norm statistics across training epochs for all four fine-tuning strategies. ResNet50 (top left), DenseNet121 (top right), EfficientNet-B0 (bottom).

4.6 Analysis

Effect of unfreezing more parameters: Across all three models, validation accuracy increases monotonically as more backbone parameters are unfrozen. The largest single gain consistently occurs between the linear probe and the first partial fine-tuning step, confirming that even a small degree of backbone adaptation yields substantial improvements. After approximately 20% of parameters are unfrozen, performance begins to plateau and full fine-tuning provides only marginal additional gains — +1.5% for DenseNet121 and +2.2% for ResNet50 over last-block fine-tuning.

Layer-wise adaptation behavior: Early layers in all three models encode low-level features such as edges, corners, and textures that are universal across domains and do not require adaptation. Deeper layers encode high-level semantic features such as object parts and scene layouts that are more task-specific. This is why unfreezing only the deepest 20% of

parameters achieves most of the benefit of full fine-tuning at a fraction of the computational cost.

Gradient norm analysis: Full fine-tuning produces significantly larger gradient norms compared to linear probe across all models. ResNet50 shows an initial gradient norm of approximately 1.74 under full fine-tuning compared to 0.84 under linear probe. EfficientNet-B0 shows the largest initial gradient norms overall (5.42 under full fine-tuning), reflecting the sensitivity of its depthwise separable convolutions to large parameter updates. Larger gradient norms indicate larger parameter updates across the network which can destabilize training if the learning rate is not carefully controlled.

Convergence stability: Linear probe converges fastest due to its small optimization space. Full fine-tuning requires more epochs to stabilize, particularly for EfficientNet-B0 where the loss curve shows larger oscillations in early epochs. DenseNet121 shows the most stable convergence across all strategies, consistent with its dense gradient flow properties.

When does fine-tuning reduce robustness? When the backbone is aggressively fine-tuned on a domain-specific dataset, the model can overwrite ImageNet’s general-purpose low-level features with features overfitted to the target distribution. This is particularly dangerous when the target dataset is small or lacks diversity. The corruption robustness results in Scenario 4 provide direct evidence of this effect, where fully fine-tuned DenseNet121 and EfficientNet-B0 collapse under Gaussian noise despite achieving clean accuracy comparable to ResNet50.

5 Scenario 3: Few-Shot Learning Analysis

5.1 Method

In this scenario, the data efficiency of each backbone was evaluated under limited supervision. The linear probe setup was used as the base configuration, and the model was retrained under three different data regimes:

- **100%** — Full training set (5,945 samples)
- **20%** — Reduced training subset (approximately 1,189 samples)
- **5%** — Extreme low-data scenario (approximately 297 samples)

All subsets were generated using a fixed random seed of 42 via deterministic NumPy permutation sampling, ensuring that all three models were trained on identical subsets. Maximum 10 epochs were used for all data regimes.

5.2 Results

Model	Data Fraction	Val Acc	Train-Val Gap (100%)
ResNet50	100%	0.7727	0.1735
	20%	0.5656	
	5%	0.1688	
DenseNet121	100%	0.8634	0.3401
	20%	0.7374	
	5%	0.4007	
EfficientNet-B0	100%	0.7414	0.0453
	20%	0.3395	
	5%	0.0711	

Table 8: Few-shot validation accuracy and train-val gap (measured at 100% data). Relative performance drops are: ResNet50 = 0.7815, DenseNet121 = 0.5358, EfficientNet-B0 = 0.9041.

The relative performance drop Δ is defined as:

$$\Delta = \frac{Acc_{100\%} - Acc_{5\%}}{Acc_{100\%}} \quad (1)$$

DenseNet121 achieves the lowest relative performance drop (53.6%), demonstrating the strongest data efficiency among the three models. EfficientNet-B0 suffers the most severe drop (90.4%), effectively failing in the 5% regime with only 7.1% validation accuracy — barely above the 3.3% random chance baseline for 30 classes.

5.3 Validation Accuracy Across Epochs

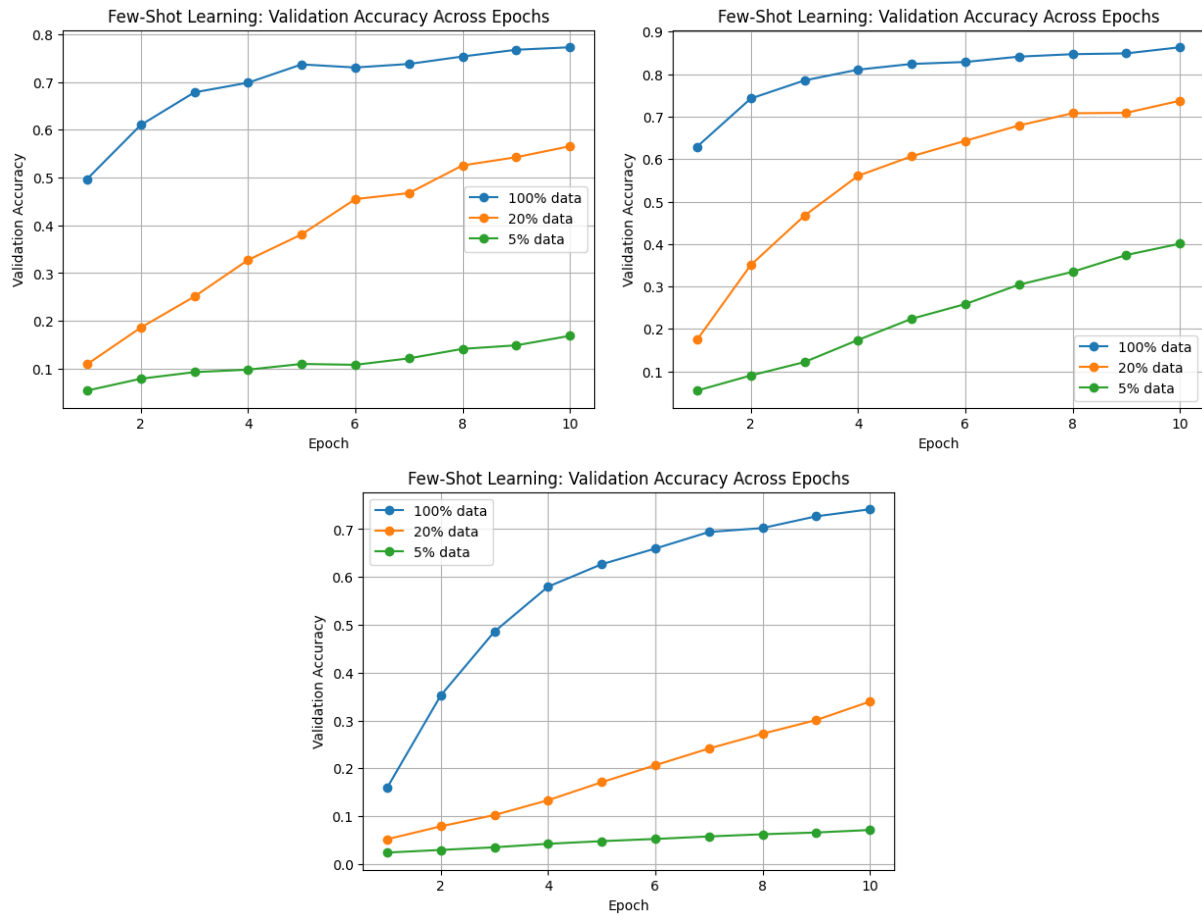


Figure 8: Validation accuracy across epochs for all three data fractions (100%, 20%, 5%). ResNet50 (top left), DenseNet121 (top right), EfficientNet-B0 (bottom).

5.4 Training–Validation Accuracy Gap

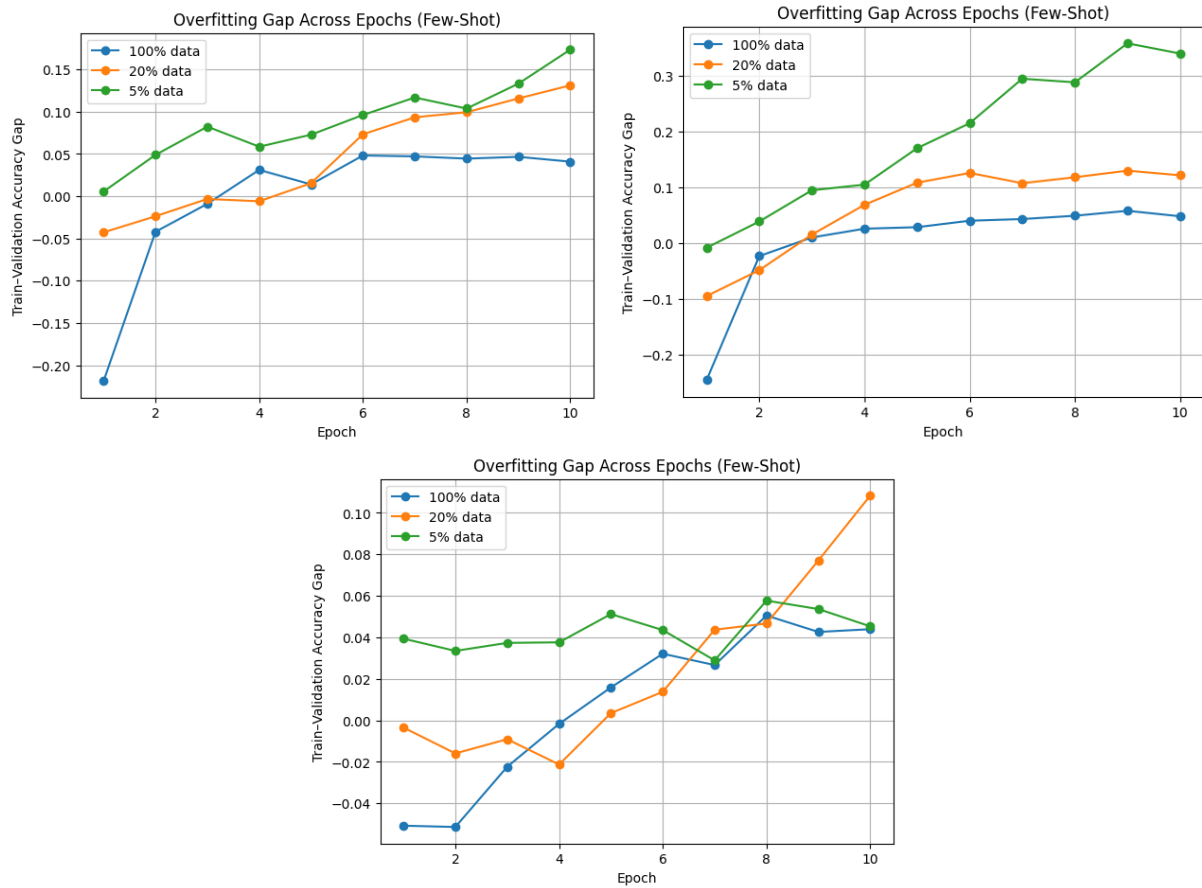


Figure 9: Training–validation accuracy gap across epochs for all three data fractions. A larger gap indicates overfitting. ResNet50 (top left), DenseNet121 (top right), EfficientNet-B0 (bottom).

5.5 Accuracy vs Data Fraction

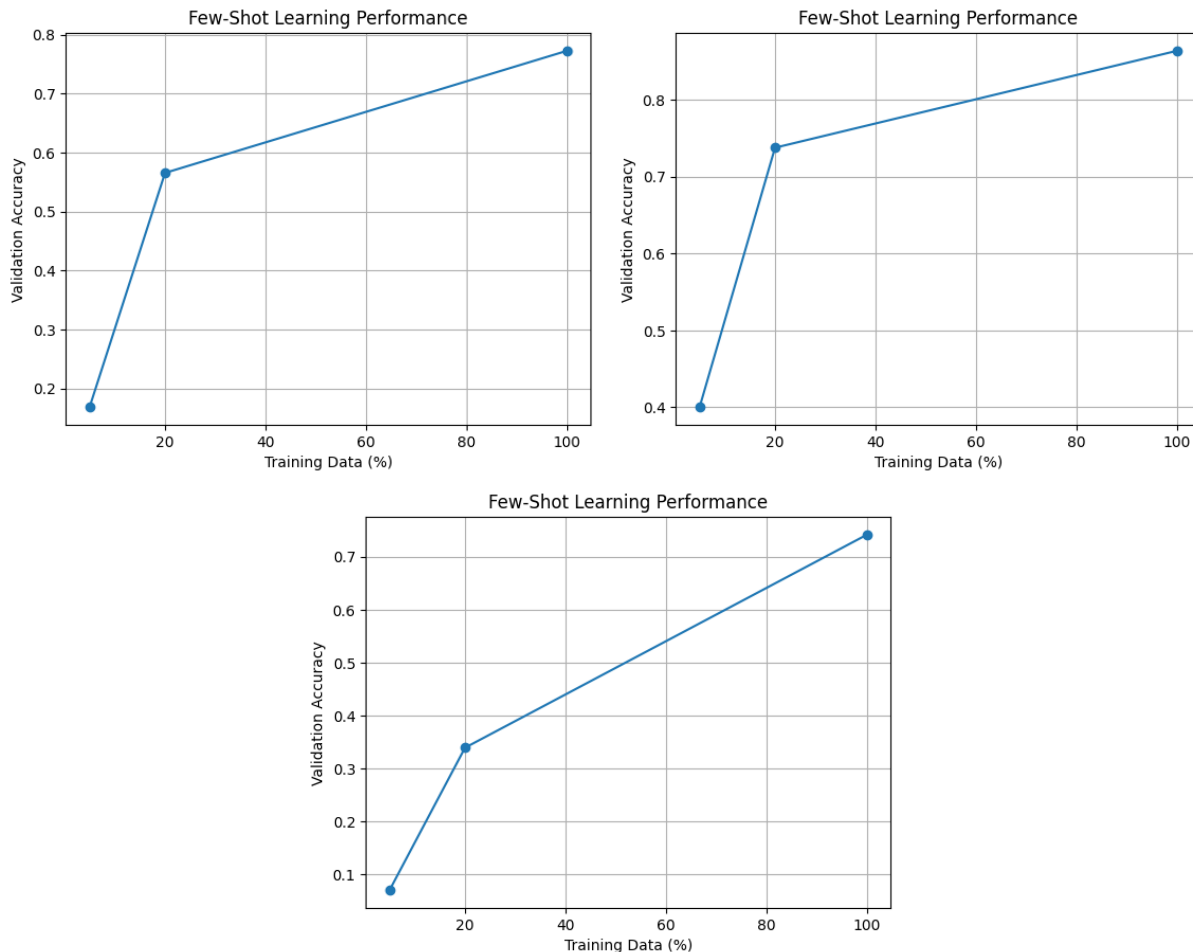


Figure 10: Validation accuracy vs training data fraction for ResNet50 (top left), DenseNet121 (top right), and EfficientNet-B0 (bottom).

5.6 Analysis

Which architecture is most data-efficient? DenseNet121 demonstrates the strongest data efficiency across all regimes. At 20% data it retains 73.7% validation accuracy, and even at 5% it achieves 40.1%. This resilience stems directly from DenseNet’s dense connectivity: every layer receives feature maps from all preceding layers, creating multiple redundant gradient pathways. With very few training samples, this structural redundancy acts as an implicit regularizer, allowing the model to generalize from limited supervision.

Which architecture is most data-hungry? EfficientNet-B0 is the most data-hungry model. Its compound-scaled depthwise separable convolutions require sufficient data to learn meaningful channel-wise filters. With only 297 training samples across 30 classes, EfficientNet-B0 struggles to build any useful representations, as evidenced by near-random accuracy (7.1%) at 5% data. Notably, the training–validation gap for EfficientNet-B0 at

100% data is only 0.045 — the smallest among the three models — which at first appears to indicate good generalization but is in fact a sign of underfitting: the model is not even memorizing the training data effectively in the low-data regime.

Overfitting behavior: As dataset size decreases, the training–validation gap increases for ResNet50 and DenseNet121, indicating classic overfitting in low-data settings. DenseNet121 shows a large gap of 0.34 at 100% data, suggesting it fits the training distribution more aggressively, but its strong validation accuracy indicates that this is productive overfitting rather than harmful memorization. ResNet50 shows a moderate gap of 0.17.

Unexpected finding: EfficientNet-B0 showed almost no overfitting (train–val gap of 0.045) even with 100% data in the linear probe setting. This is paradoxically a negative indicator — the model did not have enough representational capacity in the linear head alone to even memorize the training set, suggesting that EfficientNet-B0’s features require backbone adaptation before the linear head can be effective.

6 Scenario 4: Corruption Robustness Evaluation

6.1 Method

In this scenario, the robustness of each backbone was evaluated under controlled distribution shifts. All corruptions were applied exclusively at evaluation time on the fully fine-tuned models from Scenario 2, without any retraining or data augmentation. Three corruption types were tested:

- **Gaussian Noise** — Additive pixel-level noise with three severity levels: $\sigma = 0.05$, $\sigma = 0.1$, and $\sigma = 0.2$. Applied in the normalized image space by denormalizing, adding noise, clamping to $[0, 1]$, and renormalizing.
- **Motion Blur** — A horizontal motion blur kernel of size 5 applied via depthwise convolution across all three channels independently.
- **Brightness Shift** — Pixel intensities scaled by a factor of 1.2 after denormalization, clamped to $[0, 1]$, and renormalized.

Two metrics are reported for each corruption:

$$\text{Corruption Error} = 1 - \text{Acc}_{\text{corrupted}} \tag{2}$$

$$\text{Relative Robustness} = \frac{\text{Acc}_{\text{corrupted}}}{\text{Acc}_{\text{clean}}} \tag{3}$$

6.2 Results

Model	Corruption	Val Acc	Corruption Error	Relative Robustness
ResNet50	Clean	0.9488	0.0000	1.0000
	Gaussian $\sigma = 0.05$	0.8168	0.1832	0.8608
	Gaussian $\sigma = 0.1$	0.4761	0.5239	0.5018
	Gaussian $\sigma = 0.2$	0.0496	0.9504	0.0523
	Motion Blur	0.8655	0.1345	0.9121
	Brightness	0.9456	0.0544	0.9966
DenseNet121	Clean	0.9449	0.0000	1.0000
	Gaussian $\sigma = 0.05$	0.2891	0.7109	0.3060
	Gaussian $\sigma = 0.1$	0.0305	0.9695	0.0323
	Gaussian $\sigma = 0.2$	0.0239	0.9761	0.0252
	Motion Blur	0.8578	0.1422	0.9079
	Brightness	0.9303	0.0697	0.9846
EfficientNet-B0	Clean	0.9415	0.0000	1.0000
	Gaussian $\sigma = 0.05$	0.2395	0.7605	0.2544
	Gaussian $\sigma = 0.1$	0.0363	0.9637	0.0385
	Gaussian $\sigma = 0.2$	0.0391	0.9609	0.0416
	Motion Blur	0.8550	0.1450	0.9081
	Brightness	0.9370	0.0630	0.9953

Table 9: Corruption robustness results for all three backbones.

6.3 Analysis

Clean accuracy comparison: All three models achieve very similar clean accuracy under full fine-tuning — ResNet50 (94.9%), DenseNet121 (94.5%), and EfficientNet-B0 (94.2%). This confirms that the fine-tuning comparison in Scenario 2 is fair and that differences in robustness are architectural rather than accuracy-driven.

Gaussian noise sensitivity: ResNet50 is considerably more robust to Gaussian noise than DenseNet121 and EfficientNet-B0. At $\sigma = 0.05$, ResNet50 retains 86.1% relative robustness while DenseNet121 drops to 30.6% and EfficientNet-B0 to only 25.4%. At $\sigma = 0.1$, DenseNet121 and EfficientNet-B0 essentially collapse to near-random performance (3.2% and 3.9% relative robustness respectively) while ResNet50 still retains 50.2%.

This dramatic difference is explained by architectural characteristics:

- **ResNet50** uses standard 3×3 convolutions and residual skip connections, which help preserve stable feature representations across layers and may contribute to improved robustness under noise.
- **DenseNet121** propagates features from all earlier layers directly to later layers via dense connections. While this promotes feature reuse and efficient gradient flow, it may also increase sensitivity to perturbations introduced in early feature maps.

- **EfficientNet-B0** uses depthwise separable convolutions that process each channel independently. This makes them particularly sensitive to per-channel noise perturbations, as there is no cross-channel averaging to suppress noise.

Motion blur and brightness: All three models show strong resilience to motion blur (relative robustness $\approx 91\%$) and brightness shifts (relative robustness $> 98\%$). These corruptions do not introduce high-frequency perturbations and are therefore well-tolerated by all architectures.

Does fine-tuning reduce robustness? Yes, and the results here suggest that all three models were fully fine-tuned on AID before corruption evaluation. Despite achieving comparable clean accuracy, DenseNet121 and EfficientNet-B0 collapse under even mild Gaussian noise ($\sigma = 0.05$). Fine-tuning adapts the model to the specific pixel statistics of the training distribution, potentially overwriting the general-purpose noise-tolerant representations learned from ImageNet. ResNet50’s residual identity shortcuts appear to preserve more of these robust low-level representations even after full fine-tuning, making it the most robust architecture under distribution shift.

7 Scenario 5: Layer-Wise Feature Probing

7.1 Method

In this scenario, the quality of feature representations was examined across different depths of each backbone. Intermediate representations were extracted from three depth levels — early, middle, and final — using PyTorch forward hooks. Global average pooling was applied to the spatial dimensions of each intermediate feature map to produce a fixed-size feature vector per sample.

A logistic regression classifier with standard scaling was then trained on a fixed subset of 30 samples per class (900 samples total, seed 42) from each layer’s features, keeping the same subset consistent across all models and all layers to ensure a fair comparison.

7.1.1 Layer Selection

Model	Early Layer	Middle Layer	Final Layer
ResNet50	layer1	layer3	layer4
DenseNet121	denseblock1	denseblock3	denseblock4
EfficientNet-B0	blocks[1]	blocks[4]	blocks[-1]

Table 10: Layers selected for feature probing at each depth level.

7.2 Feature Norm Statistics

Feature norms measure the magnitude of the extracted feature vectors at each depth. Higher norms indicate stronger feature activations, while the standard deviation reflects how consistent the representations are across samples.

Model	Layer	Mean Norm	Std Norm
ResNet50	Early	18.58	0.51
	Middle	45.12	1.38
	Final	7.01	0.92
DenseNet121	Early	94.01	17.74
	Middle	27.76	1.89
	Final	28.38	0.74
EfficientNet-B0	Early	14.14	1.46
	Middle	20.45	2.62
	Final	29.27	3.04

Table 11: Feature norm statistics across early, middle, and final layers for all three backbones.

7.3 Validation Accuracy vs Depth

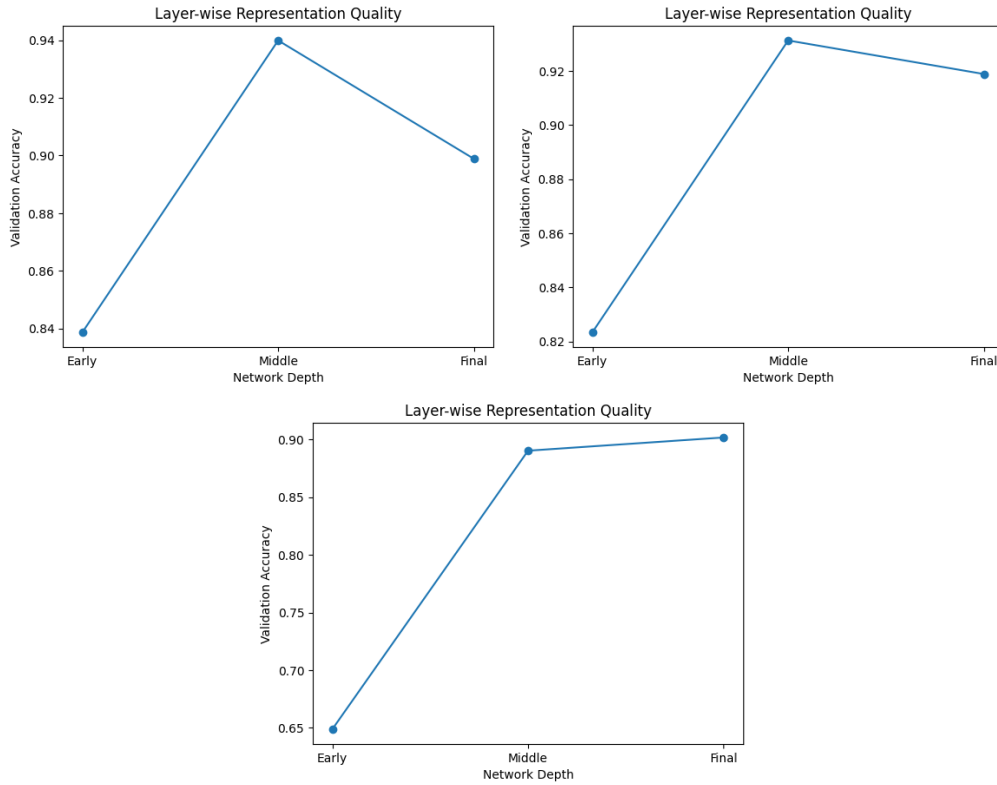


Figure 11: Linear probe validation accuracy vs network depth for ResNet50 (top left), DenseNet121 (top right), and EfficientNet-B0 (bottom). Each point represents the accuracy of a logistic regression classifier trained on features extracted from that depth level.

7.4 PCA Visualization Across Layers

7.4.1 ResNet50

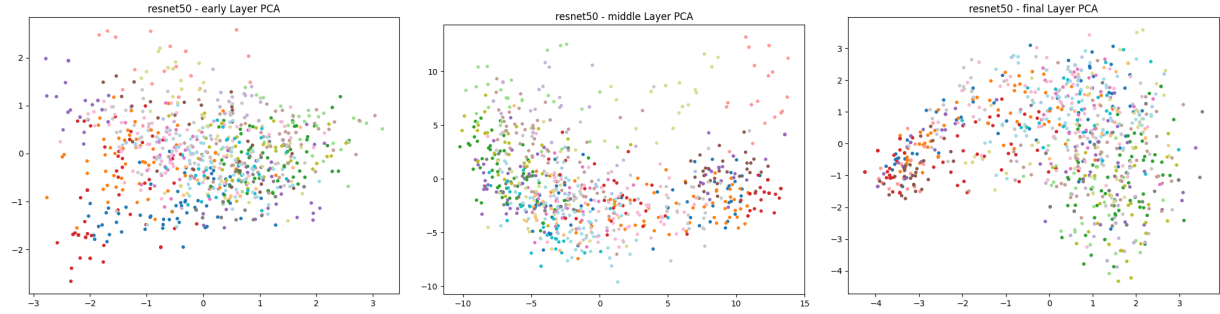


Figure 12: PCA visualization of ResNet50 features at early (left), middle (center), and final (right) layers. Each color represents one of the 30 AID classes.

7.4.2 DenseNet121

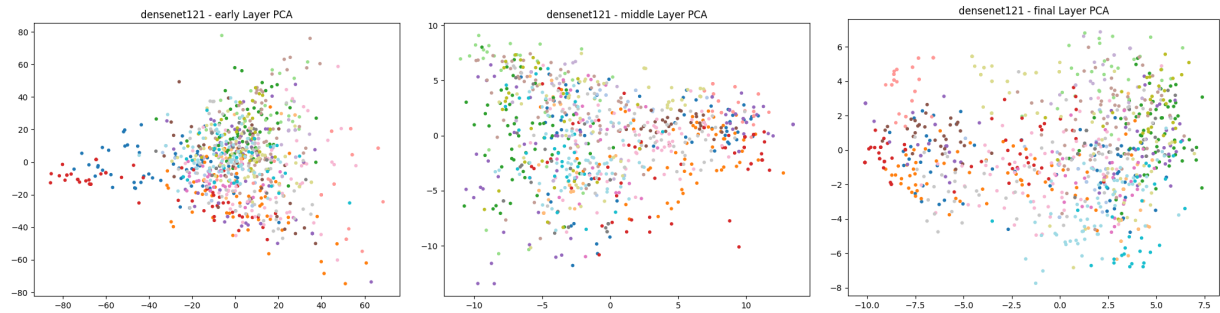


Figure 13: PCA visualization of DenseNet121 features at early (left), middle (center), and final (right) layers.

7.4.3 EfficientNet-B0

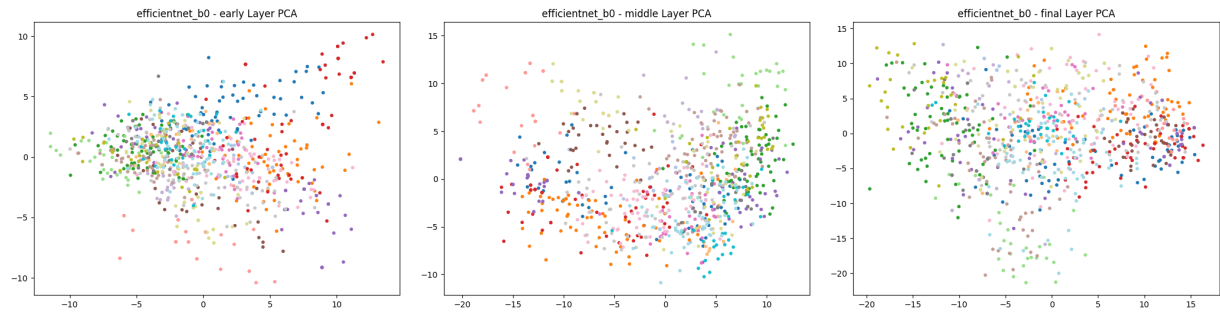


Figure 14: PCA visualization of EfficientNet-B0 features at early (left), middle (center), and final (right) layers.

7.5 Analysis

Semantic abstraction across depth: For all three models, Linear probe accuracy generally increases with depth, suggesting that deeper layers capture representations that are more useful for the aerial scene classification task.

Early layers are typically associated with low-level visual patterns, which are often less discriminative for scene-level classification. Middle layers begin to encode mid-level patterns such as structural layouts and object arrangements. Final layers produce the most compact and semantically rich representations, yielding the highest linear probe accuracy.

Feature norm patterns: The three models show distinct norm evolution patterns across depth:

- **ResNet50** shows a peak-and-refine pattern — norms increase from early (18.58) to middle (45.12) layers and then drop sharply at the final layer (7.01). This suggests that ResNet50 builds up rich intermediate representations and then compresses them into a compact discriminative vector at the final layer, which is consistent with its strong linear probe performance.
- **DenseNet121** shows an unusual pattern with a very large early norm (94.01) that drops sharply at middle (27.76) and final (28.38) layers. This is explained by DenseNet’s concatenation-based feature reuse — early denseblock features accumulate across many channels producing high-magnitude vectors, which are then distilled and compressed in subsequent blocks. The low standard deviation at the final layer (0.74) indicates very consistent and stable final representations.
- **EfficientNet-B0** shows a monotonically increasing norm pattern from early (14.14) to final (29.27) layers, consistent with its progressive feature expansion via compound scaling. The increasing standard deviation (1.46 to 3.04) indicates that final-layer features are more spread out across samples, which may contribute to its lower linear probe performance compared to DenseNet121.

PCA visualization interpretation: The PCA plots suggest progressive class cluster formation across depth for all three models. Early layer projections show largely overlapping distributions with no clear class structure. Middle layer projections show emerging clusters for visually distinct classes such as airports and storage tanks. Final layer projections produce the most compact and separated clusters, particularly for DenseNet121, which is consistent with its highest linear probe accuracy across all scenarios.

At what depth do features become aerial-specific? Based on the accuracy curves and PCA visualizations, the transition from general to representations that appear more specialized for the aerial classification task occurs primarily at the final layer for ResNet50 and EfficientNet-B0, and begins earlier at the middle layer for DenseNet121. This earlier specialization in DenseNet121 may contribute to its superior linear probe performance and data efficiency observed in Scenarios 1 and 3.

8 Discussion and Key Findings

8.1 Which Architecture Transfers Best and Why?

DenseNet121 performs the strongest overall among the evaluated backbones for transfer learning to aerial imagery in this study. It achieves the highest linear probe accuracy (88.6%), the best data efficiency under few-shot conditions, and competitive fine-tuning performance across all strategies. Under full fine-tuning, all three models converge to similar accuracy ($\approx 94\text{--}95\%$), but DenseNet121 reaches this performance more efficiently and with fewer parameters than ResNet50.

The reason lies in DenseNet121’s dense connectivity pattern. By connecting every layer to every subsequent layer within a dense block, the network is encouraged to learn compact and non-redundant representations at every depth. The final feature vector aggregates information across multiple scales and abstraction levels simultaneously, making it more directly separable by a linear classifier without requiring backbone adaptation. This multi-scale aggregation aligns well with aerial scene classification, where both low-level texture cues (vegetation, water, concrete surfaces) and high-level spatial arrangements (runway layouts, building density patterns) are jointly discriminative.

8.2 Which Model is Most Robust Under Distribution Shift?

ResNet50 appears to be the most robust architecture under distribution shift, particularly against Gaussian noise. At $\sigma = 0.05$, ResNet50 retains 86.1% relative robustness while DenseNet121 and EfficientNet-B0 drop to 30.6% and 25.4% respectively. At $\sigma = 0.1$, DenseNet121 and EfficientNet-B0 approach near-random performance while ResNet50 still retains 50.2% relative robustness.

All three models show comparable robustness to motion blur ($\approx 91\%$ relative robustness) and brightness shifts ($>98\%$ relative robustness), indicating that these corruptions are generally well-tolerated regardless of architecture. The main difference arises in sensitivity to Gaussian noise, which appears strongly influenced by how each architecture processes spatial information.

8.3 The Impact of Inductive Bias

Each architecture’s inductive bias manifests consistently across the experimental scenarios:

ResNet50 — Robustness through residual identity: ResNet50’s additive identity shortcuts help stabilize feature propagation across layers. Even under full fine-tuning on AID, these shortcuts maintain a pathway for the original ImageNet representations to influence the output. This architectural property may contribute to ResNet50’s superior robustness to noise perturbations.

DenseNet121 — Feature reuse and data efficiency: DenseNet121’s concatenation-based dense connections create a rich feature hierarchy where every layer has direct access to all preceding representations. This structural redundancy can act as an implicit regularizer in low-data settings, helping explain its strong few-shot performance. However, perturbations

introduced in early feature maps may propagate through many subsequent layers, which may increase sensitivity to input noise.

EfficientNet-B0 — Efficiency at the cost of robustness and data efficiency: EfficientNet-B0’s depthwise separable convolutions process each channel largely independently, making the model highly parameter-efficient but potentially more sensitive to low-data regimes and pixel-level noise. In the 5% few-shot setting it achieves near-random accuracy, and under $\sigma = 0.05$ Gaussian noise it loses over 74% of its clean accuracy. The Squeeze-and-Excitation modules provide channel-wise attention but cannot fully compensate for the reduced cross-channel interaction compared to standard convolutions.

8.4 When Does Fine-Tuning Degrade Robustness?

The corruption robustness results suggest that fine-tuning may reduce robustness to certain corruptions even when clean accuracy remains high. All three models achieve similar clean accuracy after full fine-tuning ($\approx 94\text{--}95\%$), yet DenseNet121 and EfficientNet-B0 show substantial degradation under mild Gaussian noise ($\sigma = 0.05$).

Fine-tuning may reduce robustness under the following conditions:

- **When the target dataset lacks diversity:** AID contains aerial imagery with relatively consistent lighting and capture conditions. Fine-tuning on this more homogeneous distribution may cause the model to specialize to the pixel statistics of aerial imagery, reducing the influence of more general representations learned from the diverse ImageNet dataset.
- **When the architecture propagates noise:** Architectures with dense skip connections (DenseNet121) or channel-independent processing (EfficientNet-B0) may be more sensitive to perturbations introduced in early layers because such perturbations can propagate through subsequent layers.
- **When fine-tuning is too aggressive:** Full fine-tuning modifies all layers, including early layers that capture general low-level patterns. Selectively fine-tuning deeper layers could preserve more robust low-level representations while still adapting high-level semantics to the target domain.

8.5 Negative and Failure Case

EfficientNet-B0 in the 5% few-shot setting represents the clearest failure case in this study. With only approximately 297 training images across 30 classes, the model achieved just 7.1% validation accuracy — only slightly above the 3.3% random chance baseline. Unlike DenseNet121 (40.1% at 5% data) or ResNet50 (16.9%), the linear classifier struggles to separate the frozen EfficientNet-B0 features under such limited supervision. The training-validation gap of only 0.045 at 100% data further suggests that the linear probe has limited capacity to adapt the EfficientNet representations without backbone fine-tuning. This highlights a critical distinction: *computational efficiency and data efficiency are not the same property.*

9 Conclusion and Computational Budget

9.1 Summary of Findings

This study evaluated three ImageNet pre-trained CNN backbones — ResNet50, DenseNet121, and EfficientNet-B0 — across five experimental scenarios on the AID aerial image classification dataset. The key findings are summarized below.

Criterion	Winner and Reason
Best linear probe transfer	DenseNet121 — achieves the highest linear probe accuracy (88.6%), indicating that its pre-trained representations transfer effectively to aerial imagery.
Best fine-tuning performance	All three models converge to similar accuracy ($\approx 94\text{--}95\%$) under full fine-tuning, though DenseNet121 achieves this with fewer parameters than ResNet50.
Most data-efficient	DenseNet121 — shows the smallest relative performance drop (53.6%) between the 100% and 5% training regimes.
Most robust to corruption	ResNet50 — retains substantially higher accuracy under Gaussian noise compared to the other architectures.
Most computationally efficient	EfficientNet-B0 — 4.05M parameters and $\approx 0.78\text{G}$ FLOPs, requiring roughly one-tenth the compute of ResNet50.
Overall best backbone	DenseNet121 — provides the best balance of representation quality, data efficiency, and fine-tuning performance across the evaluated scenarios.

Table 12: Summary of findings across all experimental scenarios.

Under linear probing, DenseNet121 achieves the highest out-of-the-box validation accuracy (88.6%), demonstrating superior feature transferability from ImageNet to aerial imagery. Under full fine-tuning, all three models converge to similar accuracy ($\approx 94\text{--}95\%$). ResNet50 emerges as the most corruption-robust architecture, retaining meaningful performance under Gaussian noise where DenseNet121 and EfficientNet-B0 collapse to near-random accuracy. EfficientNet-B0 is the most computationally efficient but the least data-efficient, failing almost completely in the 5% few-shot regime with only 7.1% validation accuracy. Overall, DenseNet121 is the strongest backbone for aerial image transfer learning, offering the best balance of representational quality, data efficiency, and fine-tuning performance.

9.2 Computational Budget

All experiments were conducted on Google Colab using an NVIDIA GPU (CUDA). The following table summarizes the approximate computational time per model per scenario.

Model	Scenario	Epochs	Approx. Time
ResNet50	Linear Probe (Scenario 1)	20	≈ 30 min
	Fine-Tuning (Scenario 2)	4 strategies $\times$ 10 epochs	≈ 60 min
	Few-Shot (Scenario 3)	3 regimes $\times$ 10 epochs	≈ 25 min
	Corruption Eval (Scenario 4)	—	≈ 5 min
DenseNet121	Linear Probe (Scenario 1)	20	≈ 25 min
	Fine-Tuning (Scenario 2)	4 strategies $\times$ 10 epochs	≈ 55 min
	Few-Shot (Scenario 3)	3 regimes $\times$ 10 epochs	≈ 22 min
	Corruption Eval (Scenario 4)	—	≈ 5 min
EfficientNet-B0	Linear Probe (Scenario 1)	20	≈ 25 min
	Fine-Tuning (Scenario 2)	4 strategies $\times$ 10 epochs	≈ 50 min
	Few-Shot (Scenario 3)	3 regimes $\times$ 10 epochs	≈ 20 min
	Corruption Eval (Scenario 4)	—	≈ 5 min

Table 13: Approximate computational time per model per scenario on a Google Colab GPU.

The total estimated compute across all three models and all scenarios is approximately **12–15 GPU hours**. All experiments remained well within the 6-hour per model per scenario budget defined in the assignment constraints.

Layer-wise probing (Scenario 5) uses logistic regression on extracted features and runs in under two minutes per model; therefore it is not included in the GPU training budget.

Sources Used

The following resources were referred to during the development of this assignment:

1. Official Documentation

- PyTorch Documentation: <https://pytorch.org/docs/stable/>
- Torchvision Documentation: <https://pytorch.org/vision/stable/>
- Timm Library (HuggingFace): <https://huggingface.co/docs/timm/>
- Scikit-learn Documentation: <https://scikit-learn.org/stable/>
- ptflops Documentation: <https://github.com/sovrasov/flops-counter.pytorch>
- UMAP Documentation: <https://umap-learn.readthedocs.io/>
- Matplotlib Documentation: <https://matplotlib.org/stable/>

2. Dataset

- Aerial Image Dataset (AID): G. Xia et al., “AID: A Benchmark Dataset for Performance Evaluation of Aerial Scene Classification,” IEEE Transactions on Geoscience and Remote Sensing, 2017. <https://captain-whu.github.io/AID/>

3. Research Papers

- He et al., “Deep Residual Learning for Image Recognition,” CVPR 2016. (ResNet50)
- Huang et al., “Densely Connected Convolutional Networks,” CVPR 2017. (DenseNet121)
- Tan and Le, “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks,” ICML 2019. (EfficientNet-B0)

4. General Programming References

- Stack Overflow — for debugging and implementation issues
- GitHub — for open source reference implementations
- Google Colab Documentation: <https://colab.research.google.com/>

5. Course Material

- Lecture slides and materials provided in GNR 638: Machine Learning for Remote Sensing II, IIT Bombay, Spring 2025–26.

6. AI Assistance

- Claude (Anthropic) — used for report writing assistance, code structuring, and documentation refinement.
- ChatGPT (OpenAI) — used for debugging support and conceptual clarification.